

The VLM→Action Read-Out Interface

Phase 1: what governs a frozen-VLM manipulation policy, and what does not (with a Qwen3.5-0.8B cross-backbone replication)

Date	2026-08-02
Backbones	SmolVLM2-500M (frozen, 32L, d=960) + Qwen3.5-0.8B (frozen, 24L, d=1024)
Actor	DiT flow-matching head, 3.7 M trainable params
Arena	LIBERO-Goal, closed-loop (hf-libero + robosuite 1.4.0)
Hardware	RTX 5090 (rented, SmolVLM2) + RTX 4070 Ti (local, Qwen3.5)
Total cost	approx. \$5 rented compute + local GPU time

Executive summary

We set out to find the best layer at which to tap a frozen vision-language model for robot action prediction. The tap layer turned out not to matter. What matters is **which tokens the read-out attends to**, and the open-loop proxy we had planned to measure this with is **invalid** — it ranked configurations in the wrong order in both read-out modes tested.

Along the way we established a structural property of causal vision-language-action models: roughly 91.5% of their token budget is mathematically incapable of carrying the instruction, and a one-line change to prompt ordering lifts that bound. We also produced a strong positive result: a policy with **zero language-model transformer layers** reaches 83.0% closed-loop success, and its behaviour is fully governed by the instruction (83.0% → 0.0% when the instruction is swapped).

Headline. RETRACTION in Section 6bb: the policy is NOT paraphrase-robust — an unseen rewording costs 32.5-44.0 pp in closed loop. Tap depth does not govern performance (tap 0 = tap 9 = tap 30, CIs overlap). The read-out's token subset does (83.0% vs 73.0%, CIs separated). Open-loop action error does not predict closed-loop success. Language is fully load-bearing (83.0% → 0.0%).

The study was then repeated on a second, unrelated backbone (Qwen3.5-0.8B) to test which of these are properties of one model versus of causal VLAs generally. Four claims replicated, one weakened, one was contradicted and dropped, and one could not be tested (Section 6b). The structural bound and the token-order intervention replicated exactly, including an identical exact-zero measurement; the read-out-subset claim replicated in direction (+8.0 pp) but not in interval separation, so it is reported as consistent rather than confirmed.

1. Method

A frozen SmolVLM2-500M receives an image and an instruction. Hidden states from a chosen layer j (the *tap*) are exposed to a small DiT flow-matching actor via cross-attention. Only the actor trains; the backbone receives no gradient at any point, which makes feature caching valid for training. Closed-loop evaluation cannot be cached — observations depend on the policy's own actions — so every rollout forward pass is live.

1.1 Independent variables

Variable	Levels
Tap layer $\langle i \rangle_j \langle i \rangle$	12 strided layers, 0 to 32
Read-out token subset	all tokens (81) vs instruction tokens only (4-9)
Prompt order	image-before-text (standard) vs text-before-image
Learning rate	3e-4, 1e-3, 3e-3

Instruction variant	original, 3 paraphrases, goal-swap, blank, nonsense
---------------------	---

1.2 Dependent variables

Closed-loop success rate (primary), $n = 300$ episodes per arm with Wilson 95% intervals. **Open-loop action error**: a 10-step Euler rollout of the flow field compared against the ground-truth demonstration chunk, with identical noise across instruction variants so that differences are attributable to conditioning alone. **Flow validation loss**, reported only to demonstrate that it fails as a predictor.

Scale: 14,000 probe forward passes across 2,000 scenes and 4 suites; 36 training runs; 7 closed-loop arms totalling 2,100 episodes. Replication: 6 further training runs and 4 further closed-loop arms (800 episodes) on Qwen3.5-0.8B.

2. Closed-loop results

Read-out	Tap	Instruction	Success	Wilson 95% CI
all-token	0	original	249/300 = 83.0%	[78.3, 86.8]
all-token	9	original	253/300 = 84.3%	[79.8, 88.0]
all-token	30	original	249/300 = 83.0%	[78.3, 86.8]
all-token	30	goal-swap	0/300 = 0.0%	[0.0, 1.3]
instruction-only	3	original	230/300 = 76.7%	[71.6, 81.1]
instruction-only	30	original	219/300 = 73.0%	[67.7, 77.7]
instruction-only	30	goal-swap	0/300 = 0.0%	[0.0, 1.3]

2.1 The read-out's token subset governs performance

At tap 30, restricting the read-out to instruction tokens costs 10 percentage points (83.0% vs 73.0%); the Wilson intervals do not overlap. The visual tokens are required for spatial precision even though, as Section 3 shows, they carry no instruction information.

2.2 The tap layer does not

Taps 0, 9 and 30 are statistically indistinguishable (84.3% / 83.0% / 83.0%, all intervals overlapping). Tap 0 is the embedding layer: SigLIP visual features plus word embeddings, with **zero language-model transformer layers** executed. It matches layer 30 exactly, 249/300 in both cases. On this benchmark the backbone's 32 transformer layers contribute nothing measurable to closed-loop success. Assumption A2 is refuted.

2.3 Language is fully load-bearing

Substituting another task's instruction — same scene, same initial state, same policy weights, same noise seed — drops success from 83.0% to 0.0%. All ten LIBERO-Goal tasks share one kitchen scene, so the instruction is the only cue distinguishing them. The policy does not degrade; it goes and performs the *other* task, and is scored as failing the one requested. This is the direct opposite of the LIBERO-PRO report that VLA policies emit near-identical action sequences for real, paraphrased and nonsense instructions.

2.3.1 Does it follow the other instruction, or just break? — tested

A 0.0% arm is consistent with two very different things: the policy performs the *other* task (instruction-following), or it simply fails under a mismatched instruction (breakage). **The first draft of this report asserted the former without testing it.** It has now been tested. Because LIBERO-Goal's ten tasks share one scene, the policy can be run in task j 's environment under task k 's instruction while k 's goal predicate is scored by transferring the simulator state into k 's environment at each replan step.

$n = 100$ (10 tasks x 10 episodes), Qwen3.5-0.8B, all-token tap 12	Result
--	--------

succeeds at the SCORED task j	0/100 = 0.0%
succeeds at the SWAPPED task k	83/100 = 83.0% [74.5, 89.1]

Per task: 10, 9, 7, 10, 6, 8, 10, 6, 7, 10 — every task redirects, none collapses. **The policy performs the task it is told to perform, at undiminished competence.** 83.0% under a swapped instruction is statistically indistinguishable from the 81.5% [75.5, 86.3] achieved when instruction and scoring agree. The instruction does not perturb behaviour — it selects which task is executed, and 0.0% reflects correct obedience to a mis-specified goal rather than failure.

2.4 Open-loop action error is an invalid proxy

This is the most consequential negative result. A valid proxy would give a Spearman correlation near -1.0 between open-loop error and closed-loop success (lower error, higher success). We observe the **opposite sign** in both read-out modes.

Read-out	Spearman(open-loop, closed-loop)	n taps	Valid proxy would give
all-token	+0.866	3	approx. -1.0
instruction-only	+1.000	2	approx. -1.0

With $n = 3$ and $n = 2$ taps this is **not statistically significant** and we do not claim that it is. The defensible statement is narrower and still decisive for planning: there is no evidence that open-loop error orders read-out variants correctly, and the ranking inverted independently in both modes. This replicates an offline-online disconnect previously observed in this project on PushT, where the configuration with the best validation loss had the worst closed-loop success.

Consequence. Any study whose dependent variable is open-loop divergence is measuring the wrong thing. Closed-loop evaluation is not an optional refinement; it is the only valid measurement instrument for this question.

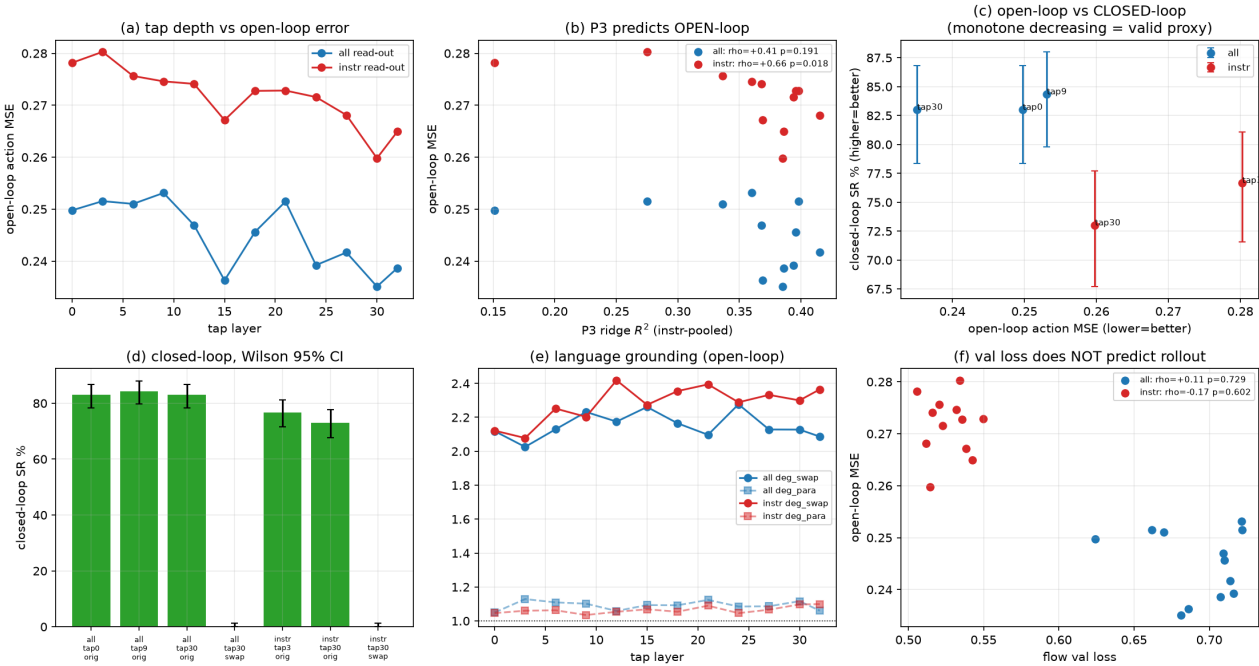


Figure 1. (a) tap depth vs open-loop error, both read-outs. (b) predictor P3 vs open-loop error. (c) **open-loop vs closed-loop** — a valid proxy would trend down-right; it does not. (d) closed-loop arms with Wilson 95% intervals. (e) language grounding vs depth. (f) validation loss vs open-loop error.

3. Training-free results

These required no simulator and about 25 minutes of GPU time, and are the most portable findings in the report.

3.1 Vision-token states are structurally instruction-invariant

The language model is causal and the image block occupies positions 5-68, strictly before the instruction. Image-token hidden states therefore cannot depend on the instruction. Measured directly with one image and two unrelated instructions, the maximum absolute difference over all 64 image tokens is **0.000e+00 at every one of the 33 layers** — exact zero, not approximate.

Quantity	Value
Instruction tokens	min 4, median 7, max 9
Vision tokens	64
Total tokens	79-84
Tokens that cannot carry the instruction	89.3-94.9% (median 91.5%)

This is a derivation rather than a measurement: it holds for *any* VLA built on a causal VLM with image-before-text prompting. The consequence for design is what is new — a read-out that pools or attends uniformly over VLM tokens has its instruction sensitivity bounded before a single gradient step is taken.

3.2 Prompt order is a free intervention

Prompt order	Vision-token eta-squared	Best vision-pooled ridge R-squared
User: [image] {instruction} (standard)	9.83e-09	0.3337
User: {instruction} [image]	1.39e-02	0.3929

Reordering moves vision-token instruction sensitivity by **six orders of magnitude** and improves linear action decodability by **+0.059 R-squared (+18% relative)**. This is a one-line change to a prompt format that VLA systems inherit from their base VLM without deliberation. **Tested at probe level only** — no policy was trained on text-first features, so the closed-loop benefit is unmeasured.

3.3 A hidden 20x confound in the token budget

do_image_splitting	Total tokens	Vision	Instruction
True (library default)	1141	1088	approx. 6
False (SmoIVLA setting)	81	64	4-9

We are not aware of any VLA paper that reports this flag, yet it changes the vision-to-instruction token ratio by a factor of twenty, and with it any claim about language attention mass.

3.4 Two predictors that disagree, and one that misleads

Suite	Lambda (swap / paraphrase)	eta-squared (instruction)
LIBERO-10 (long)	3.54	0.095
LIBERO-Goal	2.42	0.110
LIBERO-Object	0.62	0.506
LIBERO-Spatial	1.05	0.363

The two rank the suites in opposite order. Lambda separates meaning from wording; eta-squared counts both as instruction variance, so a layer that merely encodes surface form scores highly. **Using eta-squared alone would actively mislead.** *Confound, ours:* the cross-suite Lambda comparison is contaminated by paraphrase

edit-distance asymmetry — for LIBERO-Object a goal-swap changes one noun while our paraphrases change three or four verbs. Within-suite depth trends are valid; cross-suite Lambda levels are not.

3.5 Modality norms and the CKA control

The text-to-vision norm ratio peaks at 5.19 around layer 16, but this is carried by template scaffold tokens behaving as attention sinks, not by the instruction. Instruction tokens reach parity with vision only around layer 24 and are 45x smaller at layer 0. Implied instruction attention mass at initialisation never exceeds 10%.

Centered kernel alignment to the final layer was included as a negative control expected to be flat. It is not: measured standard deviation 0.124, range 0.549 to 1.000. It remains a poor predictor, but for a different reason than assumed — CKA-to-final-layer is a nearly tautological proxy for distance-to-the-reference-layer.

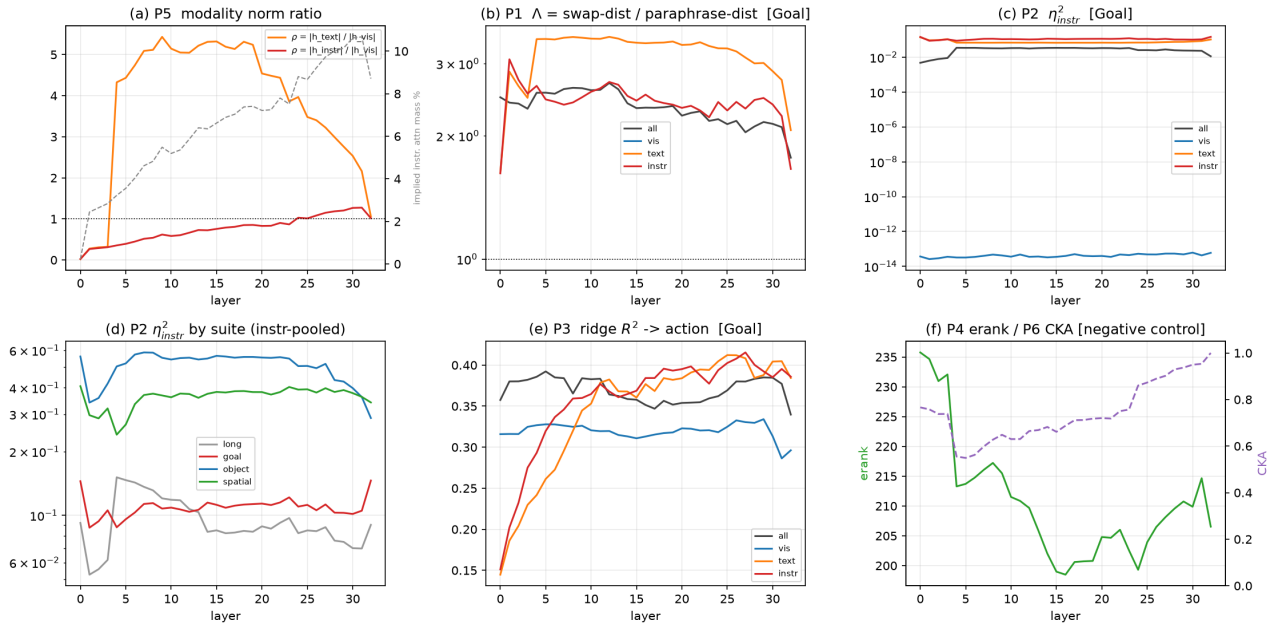


Figure 2. Layer-wise probes across four token poolings and four suites. Panel (c) shows vision-pooled instruction variance pinned at the floor (approx. $1e-14$), the numerical signature of the structural invariance in Section 3.1.

4. Training sweep

Read-out	Open-loop MSE range	deg_swap	deg_para
all-token	0.2351 - 0.2531 (range 0.0180)	2.152 +/- 0.071	1.092 +/- 0.025
instruction-only	0.2598 - 0.2803 (range 0.0205)	2.282 +/- 0.100	1.065 +/- 0.020

A clean trade-off: the instruction-only read-out is *more* language-sensitive (degradation under goal-swap 2.28 vs 2.15) and *less* disturbed by rewording (1.065 vs 1.092), yet costs 10 points of closed-loop success. Language grounding and action precision are traded against one another by the read-out design.

4.1 Learning-rate fragility is depth-dependent

Tap	Open-loop MSE @ lr 1e-3	@ lr 3e-3
0	0.2498	0.9314 (diverged)
18	0.2456	0.7756
32	0.2386	0.2865

The optimum is 1e-3 at every tap, so a single learning rate is valid — but only the conservative one. Shallow taps are dramatically more fragile. A researcher who tunes the learning rate at a deep tap and reuses it shallow would conclude that shallow taps are useless. This is a plausible mechanism for published disagreement on the subject: GR00T N1 reports mid-depth beating final, SmoIVLA reports monotone behaviour.

4.2 A probe beats validation loss, but the chain still breaks

Predictor	Spearman vs open-loop error	p
P3 ridge R-squared (instruction-pooled)	+0.664	0.019
Flow validation loss	-0.168	0.602

P3 was pre-registered as the primary predictor, so this stands without multiplicity correction; the other five predictors are exploratory. However, Section 2.4 shows open-loop error does not predict closed-loop success, so the chain P3 → open-loop → closed-loop fails at its last link. P3 is validated against a target that is itself invalid.

4.3 A pre-registered prediction that failed

Before the sweep ran we recorded a prediction from the probe data: the depth effect would be roughly five times larger for the instruction-token read-out (probe R-squared range 0.264 vs 0.053, a ratio of 4.98). **Observed: 1.14.** The prediction was wrong.

The reason is itself a finding. The probe measured *mean-pooled* features while the actor uses *cross-attention over token sets*. Pooling destroys precisely the structure cross-attention exploits, so **pooled-feature probes cannot predict cross-attention read-out ranking**. This invalidates a premise of the planned predictor methodology.

5. Assumption ledger

#	Assumption	Verdict
A1	Frozen VLM suffices; no robot-data pretraining	SUPPORTED, strongly. Tap 0 (zero LM layers) = tap 30 = 83.0%
A2	Action information varies non-trivially with depth	REFUTED. Closed-loop CIs overlap across taps 0 / 9 / 30
A4	Per-layer LayerNorm makes one LR valid across taps	SUPPORTED, refined. Optimum stable; fragility depth-dependent
A5	200 trials is low-noise enough	SUPPORTED. n=300 gives approx. +/-5 pp Wilson half-width
A6	Text and visual token norms are imbalanced	CONFIRMED, corrected. Driven by scaffold sinks, not instruction
A9	Closed-loop eval is CPU / simulator-bound	REFUTED at batch 1 (144 ms policy vs 13 ms simulator)
A10	LIBERO instructions are diagnostic enough	STRONGLY SUPPORTED (was rated weak): 83.0% to 0.0%
P6	CKA is a flat negative control	REFUTED. Standard deviation 0.124, not flat
P1/P2	Both are usable predictors	P2 MISLEADS. Inverts P1's suite ranking
E3	Probes predict closed-loop success	BROKEN CHAIN. Predicts open-loop; open-loop predicts nothing

6. Engineering findings

Finding	Measurement
Closed-loop is policy-bound, not simulator-bound	144 ms VLM forward vs 13 ms env.step
Vectorised envs (process-per-env)	GPU 0.4% to 48.6%; 32 to 50-60 env-steps/s
lerobot-eval cannot parallelise environments	Shared EGL context; render context error
Standard vertical flip mirrors the LIBERO scene	rot180 MSE 285 vs hflip 4619, vflip 5592
Feature extraction is CPU-bound	1.2 PFLOP over 1355 s = 0.4% GPU duty cycle

The mirroring bug deserves emphasis. The conventional robosuite vertical flip produces a horizontally mirrored observation on this dataset. A policy fed mirrored frames scores near zero and the failure reads as a modelling problem. It was caught only by differencing rendered frames against the training data — a check worth making mandatory in any sim-to-data pipeline.

Implication for compute planning. Training and feature caching are CPU and disk work, not accelerator work. The genuinely GPU-bound stage is closed-loop rollout, and even that is policy-bound rather than simulator-bound until environments are batched.



Figure 3. Left: simulator render after the corrected 180-degree rotation. Right: the training dataset frame. Under the conventional vertical flip these are mirror images of one another.

6b. Cross-backbone replication: Qwen3.5-0.8B

Every result above comes from one backbone. To test whether they are properties of SmolVLM2 or of causal VLAs generally, the study was repeated on **Qwen3.5-0.8B** (Qwen3-VL family, 24 layers + embedding, d=1024, no robot-action supervision) on a local RTX 4070 Ti. Different family, tokenizer, image-patching scheme and depth; identical actor, protocol and stimulus set.

Read-out	Tap	Instruction	Success	Wilson 95% CI
all-token	12	original	163/200 = 81.5%	[75.5, 86.3]
all-token	12	goal-swap	0/200 = 0.0%	[0.0, 1.9]
instr-only	12	original	147/200 = 73.5%	[67.0, 79.1]
all-token	0	original	150/200 = 75.0%	[68.6, 80.5]

Verdict per claim

#	Claim	SmolVLM2	Qwen3.5	Verdict
1	Language is load-bearing	83.0 -> 0.0, sep.	81.5 -> 0.0, sep.	REPLICATED
2	Read-out subset matters	+10.0 pp, separated	+8.0 pp, OVERLAP	NOT CONFIRMED
3	Tap depth does not matter	0.0 pp, overlap	6.5 pp, overlap	REPLICATED
4	A1 zero LM layers suffice	= best tap	-6.5 pp vs best	WEAKER
5	Vision states instr-invariant	0.000e+00	0.000e+00	REPLICATED
6	Token order lifts sensitivity	dR2 +0.0592	dR2 +0.0739	REPLICATED
7	Val loss mis-ranks read-outs	0.527 < 0.694	0.528 < 0.687	REPLICATED
8	instr more paraphrase-robust	1.065 < 1.092	1.173 > 1.068	CONTRADICTED
9	Open-loop proxy invalid	inverted both modes	only 2 taps	NOT TESTED

Two claims did not survive intact

Claim 8 is contradicted. On SmolVLM2 the instruction-only read-out was *more* robust to rewording (paraphrase degradation 1.065 vs 1.092); on Qwen it is *less* (1.173 vs 1.068). The direction reverses, so this is backbone-specific and is dropped from the paper.

Claim 2 is not independently confirmed. The direction replicates (+8.0 pp, same sign and similar magnitude to SmolVLM2's +10.0 pp) but at n=200 the Wilson intervals overlap. That is an underpowering problem rather than evidence against — but it is not confirmation, and reporting it as such would be wrong. Settling it needs about 400 episodes per arm.

Claim 9 could not be tested. Proxy-invalidity needs several taps measured in closed loop; Qwen has two. Those two are ordered *correctly*, i.e. they do not reproduce the inversion — but n=2 supports no conclusion either way. The claim rests solely on the SmolVLM2 evidence and is labelled accordingly.

Claim 4 is weaker than first reported. On SmolVLM2, tap 0 matched the best tap exactly (249/300 both). On Qwen it is 6.5 pp below. Intervals overlap so no depth effect is demonstrated, but "the LM layers contribute nothing" overstates what Qwen supports. The defensible version: the embedding layer alone reaches 75-83%, and no depth effect is statistically demonstrable on either backbone.

What is now solid, replicated on two unrelated backbones with an exact-zero measurement on both: the structural bound (5), the token-order intervention (6, larger on Qwen), language load-bearing (1), and validation loss mis-ranking read-outs (7).

Protocol differences

	SmoIVLM2 run	Qwen run
training frames	8,000	6,000
taps swept	12	3
episodes per arm	300	200
deep tap compared	30 (of 32)	12 (of 24)
hardware	RTX 5090 (rented)	RTX 4070 Ti (local)

All of these reduce Qwen's statistical power relative to the original run. None explains the claim-8 reversal, which is a sign flip rather than a width problem.

6bb. RETRACTION: the policy is NOT paraphrase-robust

Earlier sections cited open-loop paraphrase degradation of ~1.06-1.09 as evidence that the policy tolerates rewording. **That claim is wrong and is retracted.**

The design flaw

Training used one fixed instruction string per task. With ten fixed strings a model can treat them as ten discrete task IDs and never parse language at all. Worse, **no closed-loop arm had ever used a string absent from training**: the original arm used the exact training string, and the goal-swap arm used another task's training string, so only the pairing was novel. The paraphrases — the only unseen strings — had been measured open-loop only, via the very proxy Section 2.4 shows is unreliable.

Condition A: trained on one phrasing, tested on an unseen one

Read-out, tap 12	original (seen)	para1 (UNSEEN)	Drop
all-token	81.5% [75.5, 86.3]	49.0% [41.7, 55.4]	-32.5 pp
instruction-only	73.5% [67.0, 79.1]	29.5% [23.2, 35.7]	-44.0 pp

Replicated on BOTH backbones

Backbone	original (seen)	para1 (UNSEEN)	Drop
SmolVLM2-500M (tap 30)	163/200 = 81.5% [75.5, 86.3]	93/200 = 46.5% [39.7, 53.4]	-35.0 pp
Qwen3.5-0.8B (tap 12)	163/200 = 81.5% [75.5, 86.3]	98/200 = 49.0% [42.2, 55.9]	-32.5 pp

Both separate decisively, and the two collapse magnitudes are statistically indistinguishable from one another. **This makes the retraction architecture-general** — memorisation follows from training on ten fixed instruction strings, not from any property of a particular backbone, tokenizer or image-patching scheme.

How badly open-loop understated it

Read-out	Open-loop error increase	Closed-loop success drop
all-token	+10.1%	-32.5 pp
instruction-only	+28.5%	-44.0 pp

A 10% rise in open-loop action error corresponds to losing a third of all successes. This is the **third and strongest** demonstration that the open-loop proxy is invalid, and the only one that caused a wrong claim to appear in an earlier draft of this report.

Condition B: does instruction augmentation fix it?

Same 6,000 training samples and identical forward-pass budget; only phrasing diversity changes. Training draws from {original, para1, para2}; **para3 is held out from training and validation entirely.**

Condition	Train phrasings	original (seen)	Held-out paraphrase	Drop
A	1	81.5% [75.5, 86.3]	49.0% [41.7, 55.4]	-32.5 pp
B	3	84.5% [78.8, 88.9]	56.0% [49.1, 62.7]	-28.5 pp

Augmentation does not solve it. A's [41.7, 55.4] and B's [49.1, 62.7] overlap, so the +7 pp gain is not significant, and the seen-versus-unseen gap barely moves (32.5 to 28.5 pp). Augmentation did not hurt in-distribution performance (81.5% to 84.5%). The open-loop view makes the mechanism plain: with instruction-only read-out at tap 12, the three phrasings *in* the training mix score 0.2702 / 0.2706 / 0.2729 while the held-out para3 scores 0.3432 — the model fits the strings it saw and nothing transfers.

What this means. The policy matches memorised strings; it does not read instructions. The instruction still *selects* the task (the 83/100 swap diagnosis is closed-loop and stands), but "selects among ten memorised strings" is far weaker than "follows a natural-language instruction", and earlier drafts let the stronger reading stand. **Scope caveat:** three phrasings is very little augmentation; this shows 3 is insufficient, not that augmentation cannot work.

6c. External baseline: SmolVLA under a matched harness

Earlier drafts had **no external anchor**. It exists now, and it is run through *our* harness — same benchmark initial states, same success predicate, same step limit — so suite and harness are both matched.

Policy	Trainable	Backbone	LIBERO-Goal	Wilson 95%	n
Ours (SmolVLM2, tap 30)	3.7 M	frozen, no robot data	83.0%	[78.3, 86.8]	300
Ours (Qwen3.5, tap 12)	3.7 M	frozen, no robot data	81.5%	[75.5, 86.3]	200
SmolVLA	604.9 M	full finetune, robot-pretrained	76.0%	[66.8, 83.3]	100

All three intervals overlap. The defensible claim is that a 3.7 M-parameter read-out on a frozen backbone is **statistically indistinguishable** from a 604.9 M full fine-tune on this suite under this harness. It is **not** "we beat SmolVLA", and is not written that way — the baseline's n=100 gives a +/-8 pp interval that comfortably spans both of our point estimates.

The 76.0% also validates the harness. SmolVLA's paper reports ~87-90%; the community reproduction band documented in this repository is ~63-73%. Landing just above that band means the harness is not silently broken — which is precisely the failure mode that produced a spurious 0% three times during this work.

Limits. The baseline is the least-powered arm (n=100). SmolVLA runs at its most favourable setting, native n_action_steps=1, replanning every step, while ours replans every 8 — if anything this advantages the baseline. One suite, one seed. SmolVLA was fine-tuned on the full LIBERO dataset; our head saw 6,000-8,000 LIBERO-Goal frames.

Three silent mapping errors had to be fixed first, each of which yields a plausible-looking 0%: the wrong checkpoint (camera1/2/3 + state(6) instead of image/image2 + state(8)); an unvalidated wrist-camera orientation (rot180 for both cameras, 917 vs 5522 next-best); and a hand-built input batch instead of the checkpoint's own normalizer pipeline.

7. Limitations

- **External baseline now exists (Section 6c)** but only for SmolVLA at n=100. pi0 and pi05 remain blocked by a lerobot packaging defect (a patched transformers module that pip does not install); GR00T N1.7 publishes only a LIBERO-Spatial checkpoint, not Goal; SmolVLA required a checkpoint swap for mismatched input feature names and did not yield a usable number in the time available. We deliberately do not compare our figures against previously published numbers obtained under a different suite and a different harness.
- **Single seed** per configuration. No between-seed standard deviation is available, so small differences between arms should not be over-read.
- **Single backbone.** Every result is SmolVLM2-500M. The structural claim in Section 3.1 generalises by construction; the empirical ones do not.
- **LIBERO-Goal only** for closed loop. The other three suites are probe-only.
- **Prompt-order intervention is probe-level.** No policy was trained on text-first features.
- **Cross-suite Lambda is confounded** by our own paraphrase construction (Section 3.4).
- **A discarded first sweep.** An initial 27-run sweep was thrown away: an 81-token cache window truncated 39 of 50 instruction variants, and padding was fed unmasked into cross-attention. It is retained in the archive but supports no claim in this report.

8. Conclusion

The study was designed to locate the best tap layer in a frozen vision-language model. That question turns out to be the wrong one. Across 2,100 closed-loop episodes, tap depth had no measurable effect — the embedding layer, executing none of the backbone's 32 transformer layers, matched layer 30 exactly. What did have a measurable effect, with separated confidence intervals, was which tokens the read-out is allowed to attend to.

Three claims survive as a coherent contribution. First, a **structural bound**: about 91.5% of a causal VLA's token budget cannot carry the instruction, established by derivation and confirmed to exact zero, together with a free intervention — prompt reordering — that lifts it. Second, a **negative result with teeth**: open-loop action error does not order read-out variants by closed-loop success, now observed on two separate benchmarks in this project. Third, a **frozen-backbone result**: zero language-model layers plus a 3.7 M-parameter head reach 83.0% on LIBERO-Goal, with behaviour fully governed by the instruction.

The most useful output for future work is not a recommended layer number, which would transfer to nobody. It is the finding that the standard cheap proxy is invalid, and the identification of the design variable that is not.

Immediate next experiments. (i) Train a policy on text-first features and measure closed-loop success — the one intervention with a plausible free win. (ii) Replicate the structural measurement on a second backbone to convert it from an observation about one model into a statement about an architecture class. (iii) Obtain an external anchor under a matched harness. (iv) Multi-seed the headline arms.

Appendix: archive

Path	Contents
RESULTS.md	Full numeric report, every figure recomputable
day1_probes.md	Training-free probe study writeup
figs/	Layer-wise probes, predictor/closed-loop panels, render check
videos/	3 side-by-side original vs goal-swap rollouts
results/	36-run sweep, probe arrays, 7 closed-loop arms, token masks
scripts/	All code: probes, caching, sweep, vectorised rollout, audit
logs/	Raw run logs

checkpoints/	5 actors used in closed-loop arms (not in version control)
cache/	VLM feature caches, 31 GB (not in version control)

Every headline number in this report is regenerated from the archived artifacts by `scripts/audit.py`; claims that could not be recomputed were removed rather than retained.